ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ «МИСИС»

Параллельные вычисления

ЛАБОРАТОРНАЯ РАБОТА № 2

Тема: Введение в алгоритмическую и высокоуровневую оптимизацию на примере задачи умножения матриц

Группа: БПМ-22-ПО-1

Студент: Нургалиев Родион Дамирович

Ссылка на репозиторий с кодом:

https://github.com/LLrodyaLL/parallel_computing_misis_nurgaliev_r_d_bpm_22_po_1/tree/main/lab02

2026

1. Цель работы

Познакомиться с базовыми принципами выполнения алгоритмической и высокоуровневой оптимизации программного кода на примере задачи умножения квадратных матриц. Изучить влияние паттернов обращения к памяти и раскрутки цикла на производительность.

2. Описание задачи

В лабораторной работе реализованы четыре алгоритма умножения квадратных матриц размером N×N (тип элементов — float):

1. Классическое умножение матриц (classic) — тройной вложенный цикл по формуле $C[i][j] = \Sigma A[i][k] \cdot B[k][j]$.
2. Умножение с транспонированием матрицы B (transpose) — предварительно транспонируется матрица B, после чего оба обхода ведутся подряд по строкам, что улучшает использование кэша.
3. Буферизованное умножение (buffered) — столбец матрицы B копируется во временный массив tmp[], после чего умножение i-й строки A на этот столбец выполняется с последовательным доступом к памяти. Поддерживает раскрутку цикла на M итераций.
4. Блочное умножение (blocked) — матрицы разбиваются на блоки размером S×S, что позволяет повторно использовать данные, уже находящиеся в кэше. Поддерживает раскрутку цикла на M итераций.

Производительность оценивалась по формуле $P = 2N^3 / t\ (GFLOP/s)$, где $2N^3$ — объём операций с плавающей точкой (N^3 умножений и N^3 сложений).

3. Листинг программы

3.1. matrix.cpp — класс матрицы

```
#include "matrix.hpp"
#include <random>

Matrix::Matrix(std::size_t n) : n_(n), data_(n * n, 0.0f) {}

float& Matrix::at(std::size_t i, std::size_t j) { return data_[i * n_
+ j]; }
float* Matrix::row_data(std::size_t i) { return data_.data() + i * n_;
}

Matrix Matrix::random(std::size_t n, unsigned seed) {
    Matrix m(n);
    std::mt19937 gen(seed);
    std::uniform_real_distribution<float> dist(-1.0f, 1.0f);
```

```
    for (auto& x : m.data_) x = dist(gen);
    return m;
}
```

3.2. algorithms.cpp — алгоритмы умножения

```
// Классическое умножение
Matrix multiply_classic(const Matrix& A, const Matrix& B) {
    const std::size_t N = A.size(); Matrix C(N);
    for (std::size_t i = 0; i < N; ++i)
      for (std::size_t j = 0; j < N; ++j) {
        float s = 0.0f;
        for (std::size_t k = 0; k < N; ++k) s += A.at(i,k)*B.at(k,j);
        C.at(i,j) = s;
      }
    return C;
}

// Умножение с транспонированием B
Matrix multiply_transposed(const Matrix& A, const Matrix& B, bool) {
    const std::size_t N = A.size();
    Matrix BT = transpose_matrix(B); Matrix C(N);
    for (std::size_t i = 0; i < N; ++i)
      for (std::size_t j = 0; j < N; ++j) {
        float s = 0.0f;
        for (std::size_t k = 0; k < N; ++k) s += A.at(i,k)*BT.at(j,k);
        C.at(i,j) = s;
      }
    return C;
}

// Вспомогательная функция: скалярное произведение с раскруткой
static float dot_unrolled(const float* a, const float* b,
                          std::size_t N, int unroll) {
    std::vector<float> sums(unroll, 0.0f); std::size_t k = 0;
    for (; k + unroll <= N; k += unroll)
        for (int u = 0; u < unroll; ++u)
            sums[u] += a[k+u] * b[k+u];
    float s = 0.0f;
    for (float x : sums) s += x;
    for (; k < N; ++k) s += a[k] * b[k];
    return s;
}

// Буферизованное умножение (порядок циклов j, k, i)
Matrix multiply_buffered(const Matrix& A, const Matrix& B, int unroll)
{
    const std::size_t N = A.size(); Matrix C(N);
    std::vector<float> tmp(N);
    for (std::size_t j = 0; j < N; ++j) {
        for (std::size_t k = 0; k < N; ++k) tmp[k] = B.at(k,j);
```

```
        for (std::size_t i = 0; i < N; ++i)
            C.at(i,j) = dot_unrolled(A.row_data(i), tmp.data(), N,
unroll);
    }
    return C;
}

// Блочное умножение
Matrix multiply_blocked(const Matrix& A, const Matrix& B,
                        int block_size, int unroll) {
    const std::size_t N = A.size(), S = block_size;
    Matrix C(N);
    for (std::size_t ii=0; ii<N; ii+=S)
      for (std::size_t jj=0; jj<N; jj+=S)
        for (std::size_t kk=0; kk<N; kk+=S) {
          /* умножение блоков A[ii:ii+S][kk:kk+S]
             на B[kk:kk+S][jj:jj+S] с раскруткой */
          ...
        }
    return C;
}
```

3.3. benchmark.cpp — замер производительности

```
double compute_gflops(std::size_t n, double seconds) {
    const double ops = 2.0 * n * n * n;
    return ops / seconds / 1e9;
}
```

4. Сравнение конфигураций Debug и Release

Для классического алгоритма умножения матриц произведено сравнение времени выполнения в конфигурациях Debug (без оптимизации компилятора) и Release (с оптимизацией /O2).

Конфигурация	N	Время (с)	Производительность (GFLOP/s)
Debug	256	0.177127	0.189
Release	256	0.048116	0.697
Debug	512	1.882784	0.143
Release	512	0.573830	0.468

Вывод: оптимизация компилятора в конфигурации Release обеспечивает ускорение в 3,3–3,7 раза по сравнению с Debug. Для N=256 производительность возрастает с 0.189 до 0.697 GFLOP/s. Это объясняется применением компилятором оптимизаций: устранения лишних обращений к памяти, инлайнинга функций, векторизации, оптимизации работы с регистрами.

5. Результаты алгоритма с транспонированием

Алгоритм с транспонированием матрицы B обеспечивает последовательный доступ к памяти для обеих матриц, что значительно улучшает работу кэша.

N	classic (GFLOP/s)	transpose (GFLOP/s)	Ускорение
64	0.839	0.813	0.97×
128	0.736	0.839	1.14×
256	0.696	0.855	1.23×
512	0.464	0.874	1.88×
1024	0.444	0.880	1.98×
2048	0.210	0.883	4.21×

Вывод: при малых размерах матриц ($N \leq 64$) матрицы целиком помещаются в кэш, и выигрыш от транспонирования незначителен или отрицателен из-за накладных расходов на транспонирование. При $N \geq 512$, когда матрицы перестают помещаться в кэш, алгоритм с транспонированием обеспечивает устойчивое ускорение — до 4.2× при N=2048. Производительность стабилизируется на уровне ~0.88 GFLOP/s вне зависимости от N.

6. Буферизованное умножение: влияние раскрутки цикла

Буферизованное умножение копирует j-й столбец матрицы B во временный массив tmp[], после чего умножение каждой строки A на этот массив выполняется с последовательным доступом к памяти. Порядок циклов изменён на (j, k, i).

Раскрутка цикла (loop unrolling) на M итераций позволяет увеличить ILP (Instruction Level Parallelism) за счёт устранения зависимостей по данным между последовательными итерациями.

Результаты для N=256:

M (степень раскрутки)	Время (с)	Производительность (GFLOP/s)	Ускорение vs M=1
1	0.042299	0.793	1.00×
2	0.032086	1.046	1.32×
4	0.020772	1.615	2.04×
8	0.015513	2.163	2.73×
16	0.010034	3.344	4.22×

Вывод: производительность буферизованного умножения монотонно растёт с увеличением степени раскрутки M на рассмотренном диапазоне. При M=16 достигается максимум 3.344 GFLOP/s — ускорение 4.22× относительно M=1. Оптимальное значение M* = 16. Дальнейшее увеличение M теоретически может привести к переполнению кэша инструкций L1i, однако в данном диапазоне этого не наблюдается.

7. Блочное умножение: влияние размера блока S и раскрутки M

7.1. Зависимость производительности от размера блока S (M=1, N=256)

S (размер блока)	Время (с)	Производительность (GFLOP/s)
1	1.005391	0.033
2	0.554059	0.061
4	0.301759	0.111
8	0.176434	0.190
16	0.113941	0.294
32	0.089191	0.376
64	0.072814	0.461
128	0.064738	0.518
256	0.072181	0.465

Вывод: при увеличении S производительность блочного алгоритма монотонно растёт до S=128 (0.518 GFLOP/s), после чего при S=256 (весь блок = вся матрица) производительность падает до 0.465 GFLOP/s. Оптимальный размер блока S* = 128.

7.2. Влияние раскрутки M при блочном умножении (S=16, N=256)

M	Время (с)	Производительность (GFLOP/s)
1	0.112604	0.298
2	0.110104	0.305
4	0.108510	0.309
8	0.110646	0.303
16	0.117231	0.286

Вывод: для блочного алгоритма влияние раскрутки цикла незначительно. Производительность практически не изменяется с ростом M и остаётся в диапазоне 0.286–0.309 GFLOP/s. Это объясняется тем, что блочный алгоритм в данной реализации не использует транспонирование подблока B, поэтому доступ к памяти внутри блока по-прежнему не является последовательным для матрицы B. Оптимальное значение M* = 4 (незначительный максимум).

8. Сравнение алгоритмов при различных размерах матриц N

Сравнение производительности четырёх алгоритмов (с оптимальными параметрами: buffered M=4, blocked S=32 M=4) для матриц от N=4 до N=2048 в конфигурации Release:

N	classic (GFLOP/s)	transpose (GFLOP/s)	buffered M=4 (GFLOP/s)	blocked S=32 M=4 (GFLOP/s)
4	0.213	0.128	0.043	0.075
8	0.569	0.394	0.183	0.153
16	0.811	0.731	0.396	0.308
32	0.730	0.785	0.667	0.422
64	0.839	0.813	0.984	0.421
128	0.736	0.839	1.347	0.404
256	0.696	0.855	1.635	0.401
512	0.464	0.874	1.829	0.389
1024	0.444	0.880	1.927	0.395
2048	0.210	0.883	1.949	0.350

Вывод: при малых N (≤32) лучший результат показывает классический алгоритм, так как все данные помещаются в кэш и накладные расходы определяют результат. При N ≥ 64 буферизованное умножение уверенно лидирует и выходит на плато ~1.95 GFLOP/s при N ≥ 512, демонстрируя ускорение до 9.3× относительно классического алгоритма при N=2048. Блочный алгоритм в данной реализации показывает стабильно низкую производительность (~0.35–0.42 GFLOP/s) из-за неоптимального доступа к матрице B внутри блоков.

9. Выводы

1. Оптимизация компилятора (Debug → Release) даёт ускорение в 3.3–3.7× без изменения алгоритма. Это подчёркивает важность правильной конфигурации сборки для производительного кода.
2. Алгоритм с транспонированием матрицы B обеспечивает стабильное ускорение относительно классического алгоритма при N ≥ 128. При N=2048 ускорение

достигает 4.2×. Накладные расходы на транспонирование компенсируются улучшением кэш-поведения.

3. Буферизованное умножение является наилучшей реализацией: при N=2048 производительность составляет 1.949 GFLOP/s - в 9.3 раза быстрее классического. Ключевую роль играет раскрутка цикла: при M=16 производительность (N=256) достигает 3.344 GFLOP/s. Оптимальное M* = 16.
4. Блочный алгоритм в данной реализации показал неожиданно низкую производительность по сравнению с буферизованным. Причина - отсутствие транспонирования блока B при умножении, что сохраняет неоптимальный паттерн доступа к памяти. Оптимальный размер блока S* = 128, однако раскрутка цикла практически не влияет на результат (M* = 4).
5. На практике для оптимального умножения матриц рекомендуется использовать буферизованный алгоритм с раскруткой цикла или специализированные библиотеки (BLAS/OpenBLAS), применяющие SIMD-векторизацию и многоуровневое блочное разбиение.